Software Requirements Specification (SRS)
Academic Integrity Submission and Similarity Analysis System

Group 20 — Academic FBI

Darsh Kurmi — 7366180
Riya Shah — 7606627
Rishi Rakeshkumar Modi — 7606551
Rimon Paul — 7362924
Ansh Shah — 7394968
Manu Saini — 7092158
Paril Gabani — 7628753

Faculty of Mathematics and Science
Department of Computer Science
Brock University
St. Catharines, Ontario, Canada

March 14, 2026

Contents

1 Introduction **4**
1.1 Purpose 4
1.2 Scope 4
1.3 Definitions, Acronyms, and Abbreviations 4
1.4 Overview 5

2 General Description **5**
2.1 Product Perspective 5
2.2 Product Functions 5
2.3 User Characteristics 5
2.4 Operating Environment 6
2.5 Host Software Requirements and Viability 6
2.6 Assumptions and Dependencies 6

3 Functional Requirements **6**
3.1 User Accounts and Access Control . . . 6
3.2 Course and Assignment Management . . . 7
3.3 Student Submission . . . 7
3.4 Repository Management . . . 7
3.5 Privacy and Anonymization . . . 8
3.6 Similarity Analysis (Comparison Engine) . . . 8
3.7 Analysis Execution and Job Handling . . . 9
3.8 Instructor Dashboard and Reporting . . . 9
3.9 Side-by-Side Code Comparison Viewer . . . 9
3.10 Audit Logging and Activity History . . . 10
3.11 System Administration . . . 10
3.12 Error Handling and User Feedback . . . 10

4 Interface Requirements **10**
4.1 User Interfaces . . . 10
4.1.1 User Interface Components . . . 10
4.1.2 Primary Pages and Views . . . 11
4.1.3 Keyboard Shortcuts . . . 11
4.1.4 GUI Standards . . . 12
4.1.5 Usability Requirements . . . 12
4.2 Hardware Interfaces . . . 13
4.2.1 Client Devices . . . 14
4.2.2 Server Environment . . . 14
4.2.3 Repository Retention and Download . . . 14
4.3 Software Interfaces . . . 14
4.4 Communication Interfaces . . . 15

5 Performance Requirements **15**
5.1 1. Speed Requirements . . . 15
5.2 2. Response Time . . . 15
5.3 3. Throughput . . . 16
5.4 4. Real-Time Constraints . . . 16

6 Design Constraints **16**

7 Non-Functional Attributes **17**
7.1 Availability Requirements (AVAIL) . . . 17
7.2 Reliability Requirements (ROCOF) . . . 17
7.3 Resilience and Recovery Requirements . . . 17
7.4 Security Requirements (Baseline) . . . 18
7.5 Safety Requirements . . . 18
7.6 Maintainability Requirements . . . 18
7.7 Data Retention Requirements . . . 18

8 Analysis Models **19**
8.1 Use Cases 19
8.1.1 Student Use Cases 19
8.1.2 Instructor Use Cases 19
8.1.3 Administrator Use Cases 20
8.2 Entity Relationship Diagram (ERD) 21
8.3 Sequence Diagram: Student Assignment Submission 23
8.4 System Architecture 24
8.5 Similarity Analysis Pipeline (Conceptual Overview) 26

9 Conclusion **29**

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the requirements for an online code submission and similarity analysis system intended to support academic integrity verification. The document identifies system functions, interfaces, constraints, and quality attributes to guide design and implementation.

1.2 Scope

The system provides a web-based submission portal where students upload programming assignments that are stored in a repository. The system performs similarity analysis between submissions in the same offering and optionally against one or more instructor-selected secondary repositories (e.g., past offerings or curated repositories). The system presents instructors with reports and side-by-side comparisons in a user-friendly web interface while enforcing confidentiality and privacy requirements. At minimum, the system supports C, C++, and Java submissions.

1.3 Definitions, Acronyms, and Abbreviations

- **Academic Integrity:** Compliance with university rules requiring original work and proper attribution.
- **Primary Repository:** Repository containing submissions for a specific assignment offering.
- **Secondary Repository:** Additional repository used for comparison (e.g., prior offerings or instructor-defined code sets).
- **Similarity Index / Score:** A numerical value indicating the degree of similarity between two code samples.
- **Anonymization:** The process of preventing disclosure of student identity in analysis outputs by using tokenized identifiers.
- **Submission Key:** A unique identifier provided by an instructor that allows students to submit an assignment for a specific course and offering.
- **Tokenized Identifier:** A system-generated identifier used to represent a student or submission without revealing real identity information.
- **Background Job:** A long-running system task, such as similarity analysis, executed asynchronously without blocking user interaction.
- **FR:** Functional Requirement.
- **NFR:** Non-Functional Requirement.

1.4 Overview

Section 2 provides a general description of the product, users, and environment. Section 3 lists functional requirements. Section 4 describes interface requirements. Sections 5–7 define performance requirements, design constraints, and non-functional attributes.

2 General Description

2.1 Product Perspective

The product is a standalone web-based system that supports online assignment submission and similarity analysis for academic integrity review. Submissions are stored in repositories and analyzed against other submissions and optional instructor-selected repositories. The system may be hosted on any server environment suitable for the project. Clients access the system via the internet.

2.2 Product Functions

At a high level, the system shall:

- Manage users and roles (Student, Instructor, Administrator).
- Allow instructors to create courses and assignments with configurable submission rules.
- Allow students to submit assignments via a web interface using an assignment key.
- Store submissions in repositories and manage secondary repositories for comparison.
- Run similarity analysis and generate ranked results and reports.
- Provide instructor dashboards and side-by-side code comparisons.
- Protect confidentiality through anonymization and controlled disclosure of identities.

2.3 User Characteristics

- **Students:** Upload assignment files and receive submission confirmation; do not access reports or other submissions.
- **Instructors:** Configure assignments and repositories, initiate analyses, and view reports/comparisons.
- **Administrators:** Manage system-wide configuration and user/course administration.

2.4 Operating Environment

- Web application accessible through common modern browsers.
- Server-side storage for submission repositories and analysis results.
- Server-side analysis engine for similarity computation.
- Email notification capability for analysis completion.

2.5 Host Software Requirements and Viability

The system is hosted on a Virtual Private Server (VPS) environment provided by OVH. The selected hosting platform provides sufficient computational resources, persistent storage, and network connectivity to support the system's functional and non-functional requirements.

The hosting environment supports:

- Deployment of a web application accessible over HTTPS,
- Execution of asynchronous background tasks for similarity analysis,
- Persistent storage for assignment repositories, metadata, and reports,
- Secure file upload and download operations,
- Email notification services for instructors.

The use of an OVH VPS confirms the feasibility of the system deployment, as it satisfies the required performance, availability, and scalability characteristics for an academic submission and analysis platform.

2.6 Assumptions and Dependencies

- Users have internet access and a supported browser.
- Instructors provide assignment keys and submission instructions to students.
- The hosting environment provides sufficient storage and compute resources for analysis tasks.

3 Functional Requirements

3.1 User Accounts and Access Control

- **FR-1** The system shall allow users (instructors, admins) to sign in using an approved authentication method.
- **FR-2** The system shall support at least two roles: Student and Instructor. admin

- **FR-3** The system shall restrict features based on role (students cannot access instructor tools).
- **FR-4** The system shall allow an instructor to add, remove, or manage students within a course.

3.2 Course and Assignment Management

- **FR-5** The system shall allow an instructor to create a course space.
- **FR-6** The system shall allow an instructor to create an assignment within a course.
- **FR-7** The system shall allow an instructor to configure assignment settings (due date, allowed languages, file rules) and a unique key so that students can submit the assignment.
- **FR-8** The system shall have a due date for the assignment, so late submissions are not accepted.
- **FR-9** The system shall allow an instructor to view a list of assignments and their status.
- **FR-10** The instructor shall have the ability to change the due date of the assignment.

3.3 Student Submission

- **FR-10** The system shall allow a student to upload an assignment submission through the web interface.
- **FR-11** The system shall validate submissions before accepting them (file type, size, language, and assignment rules).
- **FR-12** The system shall support submissions in C, C++, and Java at a minimum.
- **FR-13** The system shall allow a submission to include multiple files (either as a zip).
- **FR-14** The system shall show a confirmation after a submission is received successfully.
- **FR-15** The system shall allow a student to resubmit before the due date if resubmissions are enabled.
- **FR-16** The system shall store each submission with an immutable timestamp.

3.4 Repository Management

- **FR-17** The system shall store accepted submissions in a primary repository for that assignment.
- **FR-18** The system shall support one or more secondary repositories (e.g., past offerings, instructor-defined repositories).

- **FR-19** The system shall allow an instructor to create and name secondary repositories.
- **FR-20** The system shall allow an instructor to upload code samples to a secondary repository.
- **FR-21** The system shall allow an instructor to select which secondary repositories to include in a similarity check.

3.5 Privacy and Anonymization

- **FR-22** The system shall assign each submission an anonymized identifier for analysis and reporting.
- **FR-23** The system shall store real student identity separately from analysis output.
- **FR-24** The system shall display anonymized identifiers in similarity reports by default.
- **FR-25** The system shall allow only authorized instructors to view the mapping between anonymized IDs and real student identities.
- **FR-26** The system shall prevent students from viewing any other student's submissions, reports, or identities.

3.6 Similarity Analysis (Comparison Engine)

- **FR-27** The system shall provide a similarity analysis feature that compares code submissions.
- **FR-28** The system shall preprocess source code prior to comparison. Preprocessing may include removal of comments, normalization of whitespace, token normalization, and filtering of instructor-provided template or boilerplate code where applicable.
- **FR-29A** The system shall compute similarity scores between code samples using fingerprint-based comparison techniques. The similarity computation may incorporate weighting mechanisms to reduce the influence of common programming patterns.
- **FR-29B** The system shall generate fingerprints from normalized source code using k-gram segmentation and fingerprint selection techniques to support efficient comparison across large repositories of submissions.
- **FR-30** The system shall compare each submission against other submissions in the same assignment repository.
- **FR-31** The system shall compare each submission against all selected secondary repositories when requested by the instructor.
- **FR-32** The system shall generate a ranked list of the most similar matches for each submission.

- **FR-33** The system shall store similarity results so they can be viewed later without re-running the analysis.
- **FR-34** The system shall allow an instructor to re-run analysis when new submissions are added or repository settings change.

3.7 Analysis Execution and Job Handling

- **FR-35** The system shall run similarity analysis as a background job (so the website remains responsive).
- **FR-36** The system shall show analysis status (queued, running, completed, failed).
- **FR-37** The system shall allow an instructor to cancel an analysis run (optional).
- **FR-38** The system shall log analysis failures with a reason message for troubleshooting.

3.8 Instructor Dashboard and Reporting

- **FR-39** The system shall provide an instructor dashboard for an assignment.
- **FR-40** The dashboard shall display a table of submissions and their highest similarity score.
- **FR-41** The dashboard shall allow sorting and filtering by similarity score, submission time, and repository source.
- **FR-42** The system shall allow an instructor to open a detailed report for a selected submission.
- **FR-43** The detailed report shall show top matches, similarity scores, and which repository each match came from.
- **FR-44** The system shall allow an instructor to export a report.
- **FR-44** The system shall allow the instructor to download the repository with all code and similarity report.

3.9 Side-by-Side Code Comparison Viewer

- **FR-45** The system shall provide a side-by-side comparison view for two code samples.
- **FR-46** The comparison view shall highlight matching regions in both samples.
- **FR-47** The comparison view shall show file names and allow switching between files for multi-file submissions.
- **FR-48** The comparison view shall allow the instructor to navigate between matches (next/previous).

3.10 Audit Logging and Activity History

- **FR-50** The system shall record key actions (submission upload, repository selection, analysis run, report view).
- **FR-51** The system shall allow instructors to view an activity history for an assignment (optional).
- **FR-52** The system shall store logs in a way that does not expose student personal information to unauthorized users.

3.11 System Administration

- **FR-53** The system shall allow an admin to manage global settings (file size limits, supported languages, retention rules).
- **FR-54** The system shall allow an admin to manage server-side repositories and backups (optional).
- **FR-55** The system shall allow an admin to disable a user account if needed (optional).

3.12 Error Handling and User Feedback

- **FR-56** The system shall show clear error messages when a submission is rejected and explain why.
- **FR-57** The system shall notify the instructor if an analysis run fails and provide a reason.
- **FR-58** The system shall prevent unauthorized access attempts and return an appropriate "access denied" message.

4 Interface Requirements

4.1 User Interfaces

The system shall provide a web-based user interface for three roles: Administrator, Instructor, and Student. The interface shall maintain a consistent layout across pages, including a navigation bar and role-appropriate options.

4.1.1 User Interface Components

The user interface shall include the following components:

- Institution selection and institution registration option.
- Role-based login for Administrator and Instructor.

- Course-based views for assignments and submissions.
- Plagiarism/similarity checking by comparing code submissions.
- An option to exclude a chunk of code from similarity comparison.

4.1.2 Primary Pages and Views

- **Home Page:** The main landing page shall include a navigation bar with *About Us*, *Why Us*, and *Help/Contact Us*. Users shall be able to select an institution from a dropdown menu. If the institution is not listed, users shall be able to register the institution.
- **Login Page:** After selecting an institution, users shall be redirected to a login page where they can log in as Administrator or Instructor. Instructors shall be provided initial login credentials by an Administrator and shall be able to set up their account by creating a password during the initial setup. Administrators shall log in using credentials provided by the university.
- **Administrator Dashboard:** After login, the Administrator shall see a dashboard showing all courses currently offered by the university in a card-based layout. When selecting a course card, the Administrator shall be able to view the instructor teaching the course and assignment submissions, including student name, student ID, and the submitted file. The Administrator shall have full visibility of the course.
- **Instructor Dashboard:** After login, the Instructor shall see a dashboard of the courses they are teaching in the current semester in a card-based layout. The Instructor shall be able to add and manage students within each course. When selecting a course, the Instructor shall be able to view assignment submission files using tokenized student identifiers by default, such that the Instructor cannot directly determine which submission belongs to which student. If permitted by role and policy, the Instructor shall be able to search for a specific student to view the corresponding submission and/or identity mapping.
- **Submission and Comparison Page:** The system shall allow the Instructor to select submitted assignments for comparison. When comparison is initiated, the system shall display a side-by-side code comparison view and highlight similar lines or logic. Beneath the comparison view, the system shall display an output panel with a similarity percentage and an explanation of highlighted code. A toggle option shall allow the Instructor to exclude a chunk of code from comparison.
- **Error Messages:** The system shall display clear and user-friendly error messages near the relevant input fields.

4.1.3 Keyboard Shortcuts

The interface shall support standard keyboard shortcuts (e.g., copy and paste using `Ctrl+C` and `Ctrl+V`).

4.1.4 GUI Standards

The system shall maintain a consistent layout and design across all pages. The navigation bar structure shall remain consistent across pages and shall include a logout option for authenticated users while they are exploring the site.

4.1.5 Usability Requirements

The system must be easy to learn and efficient to use for Administrators, Instructors, and Students, with consistent navigation, clear feedback, and accessible design.

Ease of Use

- The interface must use a simple navigation structure with core actions placed consistently (login, courses, submissions, compare).
- Each role must see only what they need:
  - Administrators: institution-wide courses and roster tools.
  - Instructors: only their courses and anonymized submissions.
  - Students: enrolled courses, assignments, and submission tools.
- The same wording and UI labels must be used throughout.
- Short helper text or tooltips must be shown for non-obvious features (e.g., tokenized IDs).

Consistency and Memorability

- Page layouts must follow the same structure across the system, even when data visibility differs by role.
- Repeated actions must behave the same way every time.
- When users return from the compare page, the system must keep their previous filters and position in the list.

Efficiency

- Submission lists must support:
  - Search (token/name/ID depending on role permissions).
  - Filters (assignment, date range, similarity threshold, status).
  - Sorting (highest similarity, newest first).
- Common workflows must require minimal steps (course → submissions → select → compare).

- The compare view must include:
  - Sync scrolling (enabled by default).
  - Next/previous match navigation.
  - A clearly visible similarity score and brief match explanation.

Error Handling and Recovery

- Upload and form inputs must be validated (file type, file size, required fields).
- Errors must be shown in plain language and include a next step (retry, change file, contact admin).
- The system must show progress/status for long operations and allow retry if the operation fails.

Accessibility

- All key workflows must be usable with a keyboard.
- Contrast and readability must be sufficient, and highlights must not rely only on color.
- The UI must work on common academic devices (laptop and tablet) without hiding critical actions.

Privacy (Instructor View)

- Instructors must see student identities in a consistent token format.
- The UI must not expose real student names/IDs in submissions, to instructors through tooltips, exports, hidden fields, or page source.
- Tokenized views must be clearly indicated to reduce confusion.

UI Guidelines

- The system must use consistent typography, spacing, and button hierarchy.
- The student portal should follow common LMS conventions (courses → assignments → submit).
- Pages must include clear titles and navigation cues.

4.2 Hardware Interfaces

This system is a web-based application and does not require any specialized hardware interfaces beyond standard client devices and a server hosting environment.

4.2.1 Client Devices

- Users (Students, Instructors, and Administrators) shall access the system using a web browser on common academic devices such as laptops and tablets.
- The system shall support standard input hardware including keyboard and mouse or trackpad.

4.2.2 Server Environment

- The system shall be hosted on a server capable of running a web application and background similarity analysis jobs.
- The server shall provide persistent storage for:
  - Uploaded assignment submission archives (e.g., ZIP files),
  - Primary and secondary code repositories,
  - Generated similarity reports and analysis metadata.

4.2.3 Repository Retention and Download

- The system shall not perform automatic repository backups.
- The system shall allow an Instructor to manually download an assignment repository as a ZIP archive for backup purposes.
- At the end of a course or configured retention period, the system shall notify the Instructor that the repository will be removed from the server.
- The system shall provide a minimum 30-day window during which the Instructor may download the repository before permanent deletion.

4.3 Software Interfaces

The system shall interface with the following external or supporting software components:

- **Authentication Service:** Manages user authentication and role verification for Administrators, Instructors, and Students.
- **Database Management System:** Stores user accounts, course data, assignment metadata, anonymized identifiers, and audit logs.
- **File Storage System:** Stores uploaded assignment archives, primary and secondary repositories, and exported reports.
- **Background Job Processing Service:** Executes similarity analysis jobs asynchronously and reports status updates to the application.

- **Email Notification Service:** Sends notification emails to instructors upon completion or failure of similarity analysis.
- **Malware/Virus Scanning Service:** Scans uploaded submission files before they are stored in the repository.

4.4 Communication Interfaces

- The system shall use HTTPS (TLS-encrypted) communication for all web-based client interactions.
- File uploads and downloads shall be performed over secure HTTP connections.
- Email notifications shall be sent using standard email protocols (e.g., SMTP or a secure email API).
- Internal communication between the web application and background analysis services shall use secure and reliable inter-process communication mechanisms.
- The system shall enforce appropriate timeouts and error handling for failed or interrupted network communications.

5 Performance Requirements

5.1 1. Speed Requirements

- The system shall authenticate users and load the initial dashboard within a reasonable time under normal operating conditions.
- The system shall accept and validate assignment submission uploads without noticeable delay for typical assignment file sizes.
- Similarity analysis execution time may vary based on the number and size of submissions; therefore, the system shall not impose a fixed upper bound on analysis execution time.

5.2 2. Response Time

- The system shall provide immediate feedback for user-initiated actions such as login attempts, navigation between pages, and submission validation.
- For long-running operations such as similarity analysis, the system shall update the analysis status (e.g., queued, running, completed, failed) without requiring the user to remain on the page.
- Upon completion or failure of a similarity analysis run, the system shall notify the instructor via email that the results are available.

5.3 3. Throughput

- The system shall support multiple concurrent users accessing the web interface, including simultaneous student submissions near assignment deadlines.
- The system shall support processing multiple analysis jobs sequentially or concurrently, subject to available server resources.

5.4 4. Real-Time Constraints

- The system has no hard real-time constraints.
- Similarity analysis shall be performed asynchronously as a background process and shall not block normal user interaction with the web interface.
- The system shall ensure that completion of background analysis jobs is reliably communicated to the instructor through status updates and email notifications.

6 Design Constraints

- The system shall be implemented as a web-based application accessible through standard web browsers.
- The system shall support programming language submissions in C, C++, and Java at a minimum.
- The system shall not execute submitted code at any time to prevent security risks.
- The system shall perform malware and security checks on all uploaded submissions before storing them in any repository.
- The system shall anonymize student submissions for similarity analysis and reporting, and shall store identity mappings separately from analysis results.
- The system shall enforce role-based access control, ensuring that users can access only features and data permitted by their assigned role.
- The system shall execute similarity analysis as an asynchronous background process to avoid blocking normal user interaction.
- The system shall retain submission repositories for a limited, configurable period (e.g., duration of a course) and shall delete them after expiration unless the Instructor explicitly downloads them.
- The system shall comply with applicable academic integrity policies and data protection regulations governing the handling of student information.
- The system shall be designed such that repository storage, similarity analysis, and reporting components can operate independently to support maintainability and scalability.

7 Non-Functional Attributes

7.1 Availability Requirements (AVAIL)

Availability requirements describe the extent to which the system is operational and accessible when required.

- **NFR-AV1** The system shall be available to Students, Instructors, and Administrators during normal academic usage periods, excluding scheduled maintenance windows.
- **NFR-AV2** The system shall allow users to view previously submitted assignments, repositories, and completed similarity reports while background similarity analyses are executing.
- **NFR-AV3** If the system becomes temporarily unavailable, it shall present a clear status message indicating that the service is unavailable.
- **NFR-AV4** After an assignment due date and time, the submission portal shall remain available but shall prevent further uploads and display a clear "submissions closed" message.

7.2 Reliability Requirements (ROCOF)

Reliability requirements specify constraints on the rate of occurrence of failures (ROCOF) during normal system operation.

- **NFR-RO1** The system shall minimize the rate of user-visible failures, including failed uploads, lost submissions, and failed similarity analysis executions.
- **NFR-RO2** The system shall ensure atomic submission handling such that a submission is either fully accepted and stored or not stored at all.
- **NFR-RO3** The system shall not lose accepted submissions or completed similarity analysis results due to internal software failures.
- **NFR-RO4** If a similarity analysis job fails, the system shall record the failure reason and allow the instructor to re-run the analysis without data loss.

7.3 Resilience and Recovery Requirements

Resilience requirements describe the system's ability to recover from failures and continue operation.

- **NFR-RE1** Similarity analysis shall be executed asynchronously as a background process so that the web interface remains usable during analysis.
- **NFR-RE2** The system shall track the execution state of background analysis jobs (e.g., queued, running, completed, failed) to prevent silent job loss.
- **NFR-RE3** In the event of a server restart or transient failure, the system shall preserve submission data and analysis job state where possible.

7.4 Security Requirements (Baseline)

Security requirements define baseline protections appropriate for an academic system without assuming high-security deployment.

- **NFR-S1** The system shall enforce role-based access control such that users may access only the data and features permitted by their role.
- **NFR-S2** Student submissions shall be anonymized for similarity analysis and instructor-facing reports using tokenized identifiers by default.
- **NFR-S3** Access to the mapping between tokenized identifiers and real student identities shall be restricted to authorized users only.
- **NFR-S4** The system shall transmit user data over secure communication channels (e.g., HTTPS).

7.5 Safety Requirements

Safety requirements address protection against harmful or unintended system behavior.

- **NFR-SA1** Submitted code shall not be executed on the server at any time.
- **NFR-SA2** The system shall scan uploaded submission files for malware or harmful content before storing them in any repository.
- **NFR-SA3** Submissions containing unsupported file types or flagged as malicious shall be rejected and not stored.

7.6 Maintainability Requirements

Maintainability requirements ensure that the system can be modified and maintained over time.

- **NFR-M1** The system shall log operational errors and analysis failures to support debugging and maintenance.
- **NFR-M2** The system shall separate user interface, repository management, and similarity analysis components to support independent maintenance.
- **NFR-M3** Configuration parameters such as supported languages, file size limits, and retention periods shall be adjustable without modifying source code.

7.7 Data Retention Requirements

- **NFR-D1** The system shall retain repositories for a limited, configurable duration (e.g., the duration of a course).
- **NFR-D2** The system shall notify the instructor prior to repository deletion and provide at least a 30-day window to download the repository as a ZIP archive.

8 Analysis Models

8.1 Use Cases

This section summarizes the primary use cases supported by the system. These use cases describe how Students, Instructors, and Administrators interact with the system to submit assignments, manage course content, and review similarity analysis results.

8.1.1 Student Use Cases

- **UC-S1: Select Institution** — The student selects their institution from the system.
- **UC-S2: Verify Assignment Key** — The student enters a provided assignment key and the system validates it.
- **UC-S3: Upload Submission** — The student uploads an assignment archive (ZIP) and receives confirmation or rejection with reasons.
- **UC-S4: Resubmit Submission (Optional)** — The student uploads a revised submission before the due date if resubmission is enabled.

8.1.2 Instructor Use Cases

- **UC-I1: Create and Manage Course** — The instructor creates a course and manages enrolled students.
- **UC-I2: Create and Configure Assignment** — The instructor creates an assignment, sets rules (languages, due date, file constraints), and generates a unique submission key.
- **UC-I3: Manage Repositories** — The instructor creates or uploads secondary repositories and selects which repositories to include for comparison.
- **UC-I4: Start Similarity Analysis** — The instructor initiates similarity analysis for an assignment submission set.
- **UC-I5: View Similarity Reports** — The instructor views ranked similarity results and opens detailed reports.
- **UC-I6: Compare Two Submissions** — The instructor views a side-by-side comparison with highlighted matched regions.
- **UC-I7: Export Reports / Download Repository** — The instructor exports similarity reports and downloads repositories for backup before retention expiry.
- **UC-I8: Search Student (Restricted)** — If the instructor wishes to do so, the instructor searches for a student to resolve tokenized identifiers to a real identity.

8.1.3 Administrator Use Cases

- **UC-A1: Manage Instructors and Courses** — The administrator creates, updates, or disables instructor accounts and manages institution-wide courses.
- **UC-A2: Configure Global System Settings** — The administrator configures limits and rules (e.g., file size, supported languages, retention period).
- **UC-A3: View Institution Activity** — The administrator reviews system-level logs and activity summaries without violating privacy rules.

8.2 Entity Relationship Diagram (ERD)

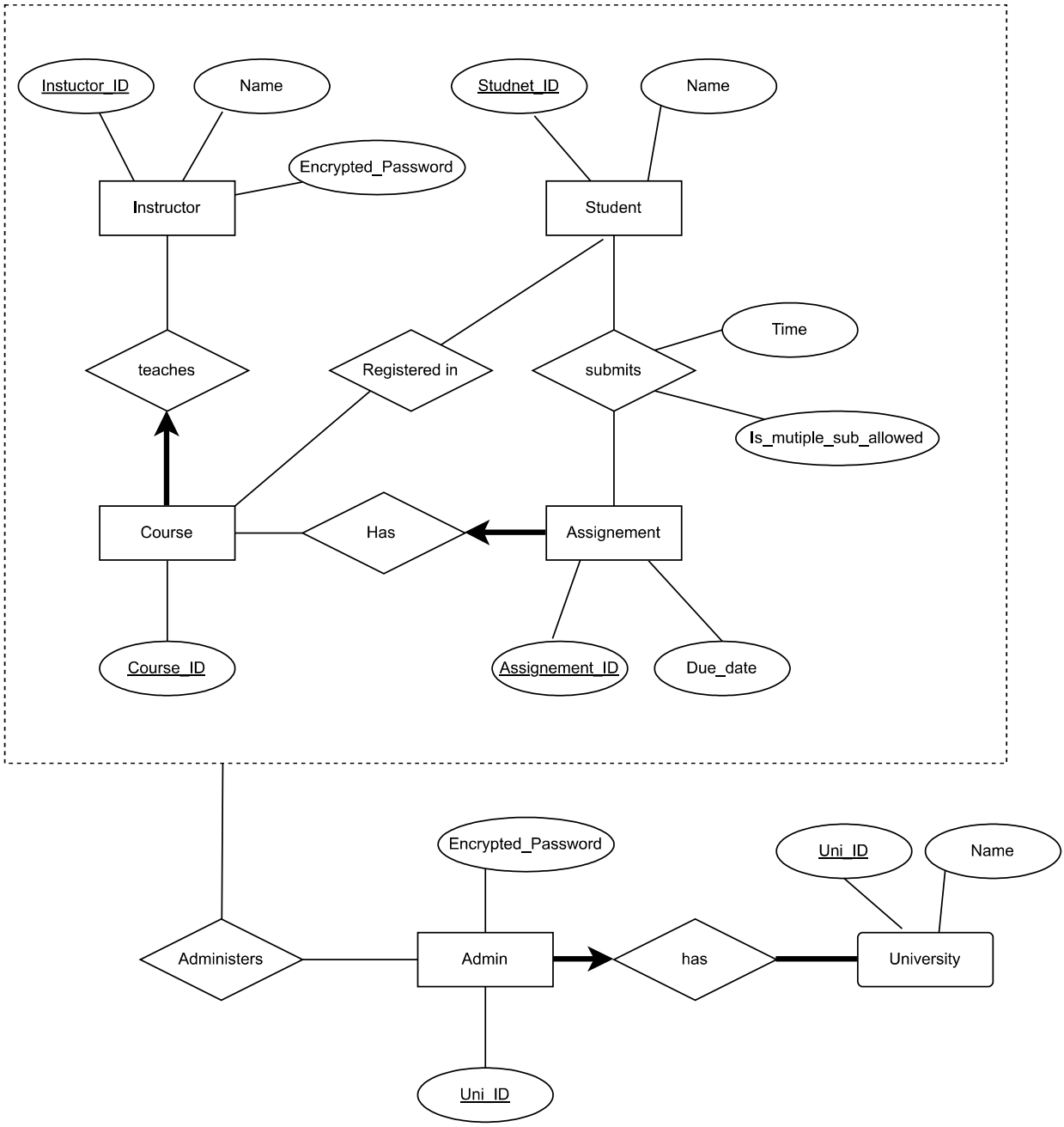

Figure 1: Entity-Relationship diagram

- The ERD shown in Figure 1 models the core entities involved in the system, including **University**, **Admin**, **Instructor**, **Student**, **Course**, and **Assignment**.
- A **University** may haveone or more **Admins**, who oversee system configuration and management at the institutional level. Admins provide Instructor(s) with ID and password. They can add/remove instructors, course assignments and students.
- **Instructors** are associated with **Courses** through a teaching relationship.
- **Students** are registered in courses and submit assignments associated with those courses.
- Each **Course** may contain multiple **Assignments**, each with its own due date.
- Assignment submissions record metadata such as submission time and resubmission allowance.
- User entities store identifying information and encrypted credentials to support secure authentication.

8.3 Sequence Diagram: Student Assignment Submission

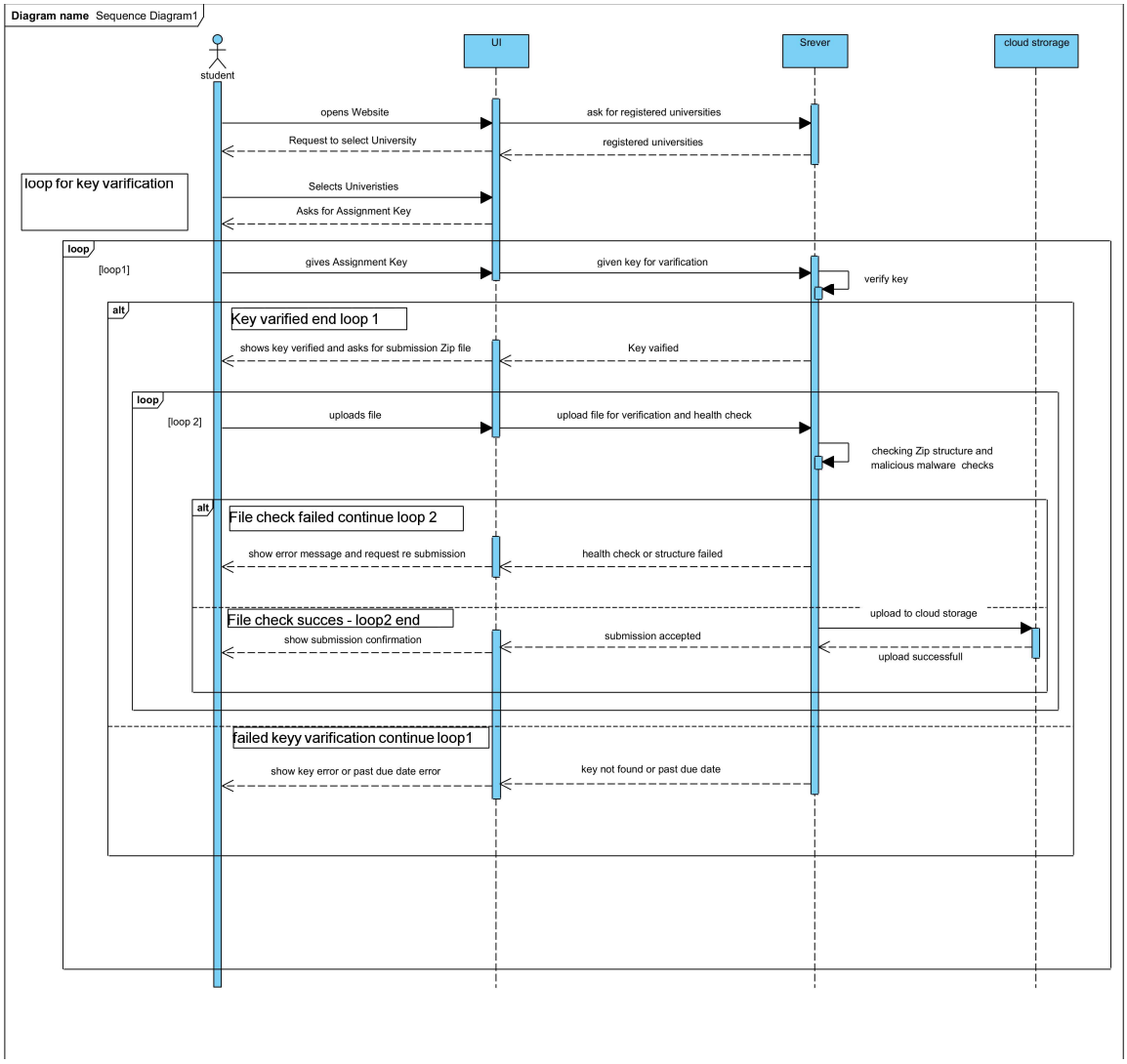

Figure 2: Student Assignment Submission sequence diagram

This sequence diagram in Figure 2 illustrates the interaction flow between a Student, the web-based User Interface (UI), the Server, and Cloud Storage during the assignment submission process.

- The Student accesses the system through the web interface and selects an institution from the list of registered universities retrieved from the Server.
- The UI prompts the Student to enter an assignment submission key. The key is sent to the Server for validation.
- The Server verifies the assignment key to ensure that it exists, is associated with the selected institution, and that the assignment is still open for submission.

- If the assignment key is invalid or the submission deadline has passed, the Server returns an error message and the UI prompts the Student to re-enter a valid key.
- Upon successful key verification, the UI enables the assignment submission interface and allows the Student to upload a submission archive.
- The uploaded submission is forwarded to the Server, which performs validation checks including file structure verification and malware or health checks.
- If the submission fails validation, the Server rejects the submission and the UI displays an appropriate error message, allowing the Student to resubmit.
- If the submission passes all validation checks, the Server accepts the submission and stores it in cloud-based storage.
- Once storage is successful, the Server confirms submission completion and the UI displays a submission confirmation message to the Student.

8.4 System Architecture

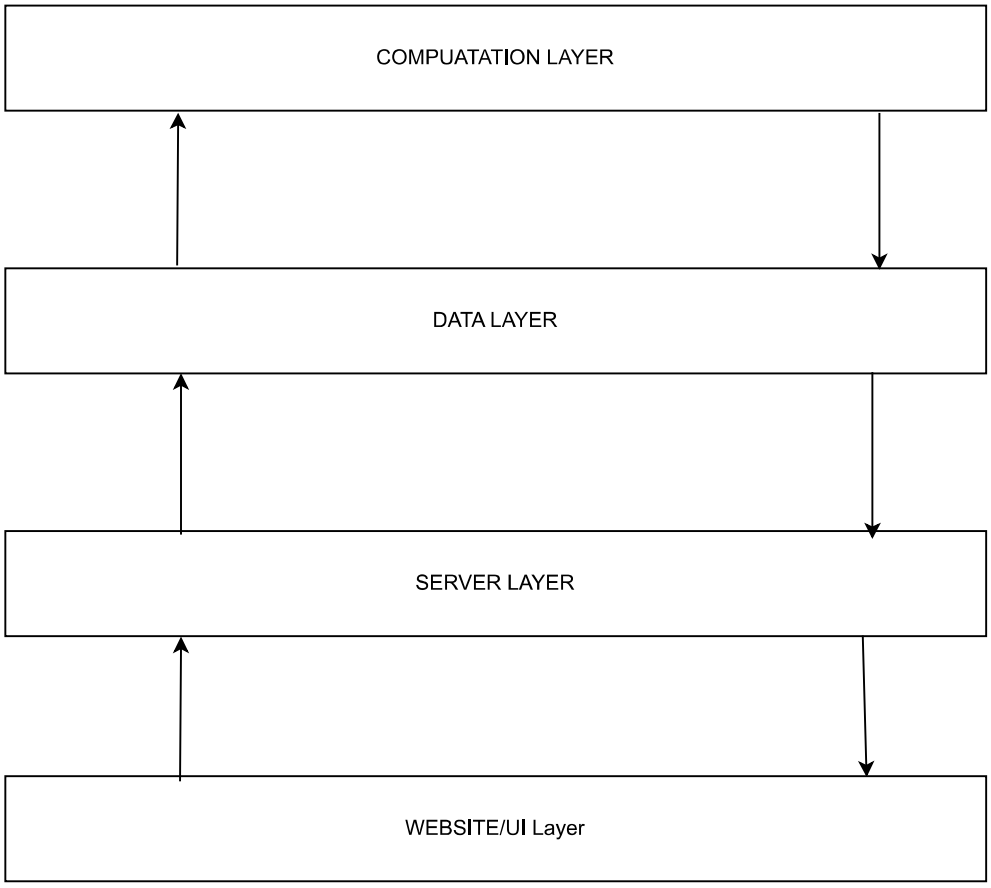

Figure 3: Layered Architecture Diagram

The system follows a modular, service-oriented Layered architecture designed to support secure assignment submission, asynchronous similarity analysis, and controlled access to academic data. Figure 3 illustrates the high-level architecture of the student assignment submission and similarity analysis system.

The **User Interface (UI) Layer** provides the web-based interface used by Students, Instructors, and Administrators. Students interact with this layer to select their institution, verify assignment keys, upload assignment submissions, and receive submission confirmations. Instructors and Administrators use the interface to manage courses, assignments, repositories, and to view similarity analysis results and reports. The UI layer enforces role-based visibility, ensuring that users only see information appropriate to their role.

The **Application Server Layer** serves as the central control layer of the system. It handles authentication and authorization, validates user requests, manages courses and assignments, and enforces submission rules such as due dates and file constraints. When a submission is received, the server performs structural validation and security checks, assigns an anonymized submission identifier, and ensures that student identity data is separated from analysis data. All role-based access restrictions and privacy controls are enforced within this layer.

The **Storage Layer** provides persistent storage for system data and artifacts. This includes user accounts, course and assignment metadata, anonymized submission identifiers, similarity results, and system logs. Source code submissions and repository archives are stored separately from identity-related data to maintain confidentiality and comply with data protection requirements. Repositories are retained for a limited duration and may be downloaded by instructors before scheduled deletion.

The **Background Analysis Layer** is responsible for executing similarity analysis tasks asynchronously. Similarity computation is triggered explicitly by the instructor and runs independently of the web interface to ensure responsiveness. This layer processes submissions using a custom similarity comparison engine that supports C, C++, and Java source code. Submissions are compared against other submissions in the same assignment as well as instructor-selected secondary repositories. The analysis produces similarity scores and matched code regions, which are returned to the server for storage and reporting.

The similarity analysis engine performs multiple internal stages of processing to enable efficient large-scale comparison of source code submissions. After normalization and fingerprint generation, the engine constructs internal data structures that map fingerprints to the submissions in which they appear. These structures allow the system to identify candidate submission pairs that share structural similarities before performing detailed similarity calculations. By reducing unnecessary pairwise comparisons and prioritizing meaningful matches, the engine improves scalability and enables efficient analysis of large submission repositories. The comparison process also incorporates weighting strategies and block reconstruction techniques to identify and highlight structurally similar code segments in the final analysis results.

This architectural design ensures separation of concerns, protects sensitive academic data, supports asynchronous processing for long-running analysis tasks, and allows the system to

scale and evolve while maintaining compliance with academic integrity and privacy requirements.

8.5 Similarity Analysis Pipeline (Conceptual Overview)

The similarity analysis engine processes submitted source code without executing it and is designed to detect structural and logical similarity even when superficial modifications are applied. The analysis pipeline transforms raw source code into normalized representations, generates fingerprints representing structural code patterns, and compares these fingerprints across submissions to identify suspicious similarity.

The pipeline is composed of the following stages.

- **Input Acquisition:** Submitted source code files are retrieved from the assignment repository after validation and anonymization have been completed. Each submission is associated with a unique anonymized identifier so that similarity analysis can be performed without exposing student identities.

  Submissions may consist of multiple source files contained within an archive. Each source file is extracted and processed individually while maintaining references to the originating submission.

- **Lexical and Structural Parsing:** Source code is parsed into a token-based representation suitable for structural comparison. The parsing stage converts raw program text into a sequence of lexical tokens representing programming constructs such as keywords, operators, control-flow statements, and identifiers.

  This transformation removes superficial formatting differences while preserving program structure and execution logic. The token stream serves as the canonical representation used for subsequent similarity analysis stages.

- **Code Normalization:** Normalization reduces noise introduced by superficial code modifications. The engine applies several normalization strategies, including:

  - removal of comments and non-functional whitespace,
  - normalization of identifier names and literals,
  - structural abstraction of syntactic constructs where appropriate.

  These transformations allow the system to detect similarity even when variable names are changed, comments are removed, or formatting is altered.

  The resulting normalized token stream provides a consistent structural representation of the program independent of stylistic differences.

- **Boilerplate and Template Code Filtering:** Assignments often include instructor-provided starter code or template files that appear in all submissions. Including this shared code in similarity calculations would artificially inflate similarity scores.

To address this issue, the system processes instructor-provided template files using the same fingerprint generation pipeline applied to student submissions. Fingerprints derived from these templates are stored in a boilerplate reference set.

During analysis, fingerprints from student submissions that match boilerplate fingerprints are excluded from similarity comparisons. This ensures that similarity scores primarily reflect student-authored logic rather than shared assignment scaffolding.

- **Feature Extraction via Adaptive k-Gram Segmentation:** The normalized token stream is segmented into overlapping sequences of tokens known as k-grams. Each k-gram represents a contiguous structural unit of code.

  Rather than using a fixed value of k, the engine may adjust the k-gram length based on the size of the input token sequence. Adaptive k-gram selection improves fingerprint stability across submissions of varying lengths by maintaining an appropriate level of structural granularity.

  This segmentation converts the program structure into a series of comparable units suitable for hashing and fingerprint generation.

- **Fingerprint Generation using Winnowing:** Each generated k-gram is converted into a hash value representing that structural unit of code. Because the number of k-grams can be large, the engine applies a fingerprint selection strategy based on the winnowing algorithm.

  Winnowing selects representative hashes from sliding windows across the hash sequence, producing a compact set of fingerprints that preserve significant structural patterns while discarding redundant hashes.

  This approach provides two important benefits:

  - reduced storage and computational overhead,
  - guaranteed detection of matching regions longer than a minimum threshold.

  Each fingerprint retains metadata describing its originating file and the source code line range associated with the fingerprint.

- **Candidate Pair Identification via Inverted Indexing:**Directly comparing every pair of submissions would result in quadratic computational complexity and would not scale efficiently as class sizes increase.

  To address this challenge, the engine constructs an inverted index mapping each fingerprint hash to the set of submissions that contain that fingerprint. Using this index, the system identifies candidate submission pairs that share at least one fingerprint.

  Only these candidate pairs are subjected to detailed similarity computation. This significantly reduces the number of comparisons required while maintaining detection accuracy.

- **Weighted Similarity Computation:**For each candidate pair, the engine compares their fingerprint sets to compute a similarity score. The system uses a fingerprint-based set similarity metric derived from the Jaccard coefficient.

To improve robustness, the engine incorporates frequency-based weighting. Fingerprints that appear frequently across many submissions represent common programming constructs and therefore contribute less to the similarity score. Rare fingerprints that appear in only a small number of submissions contribute more strongly.

This weighting mechanism reduces the influence of widely occurring programming patterns (such as common loop constructs) and emphasizes unique structural similarities between programs.

- **Match Region Reconstruction:**Fingerprint matches between two submissions are mapped back to their corresponding source code locations. Adjacent or overlapping fingerprint matches are grouped together to reconstruct contiguous code segments exhibiting similarity.

  This reconstruction step produces meaningful matched regions rather than isolated fingerprint matches. These regions represent segments of code that share structural similarity across submissions.

- **Block Density Analysis and Confidence Classification:**Each reconstructed matched region is analyzed to determine its structural significance. The engine calculates the density of fingerprint matches within the region relative to its total length.

  Regions with high match density indicate strong structural similarity and are classified as high-confidence matches, while regions with lower density are classified as medium or low confidence.

  This classification assists instructors in distinguishing between incidental similarities and segments that may indicate substantial code sharing.

- **Severity Scoring and Ranking:** To prioritize instructor review, the system calculates an overall severity score for each pair of submissions. The severity score combines multiple indicators, including:

  - overall similarity score,
  - density of matched regions,
  - proportion of high-confidence blocks.

  Submission pairs are ranked by severity score so that the most suspicious similarities appear first in instructor reports.

- **Result Aggregation and Reporting:** The final stage aggregates similarity results and prepares them for storage and presentation. For each flagged submission pair, the system records:

  - similarity scores,
  - matched source code regions,
  - file locations and line ranges,
  - confidence classifications for detected blocks.

These results are stored in the analysis repository and are later accessed by instructor-facing reporting tools, including ranked similarity dashboards and side-by-side code comparison views.

This conceptual pipeline enables the system to detect plagiarism attempts involving variable renaming, reordered statements, and equivalent control-flow transformations.

9 Conclusion

This Software Requirements Specification (SRS) defines the requirements for a web-based academic integrity verification system that supports online assignment submission, repository management, and similarity analysis. The document specified functional capabilities for user and role management, course and assignment configuration, secure student submission handling, anonymized reporting, and instructor-facing comparison and dashboard tools. It also detailed interface requirements, performance expectations, design constraints, and key non-functional attributes, including availability (AVAIL), reliability (ROCOF), resilience, baseline security, safety controls, and data retention policies.

The analysis models (ERD, sequence diagram, and architecture diagram) capture the core data relationships and system interactions required to implement the solution while maintaining confidentiality and enforcing role-based access. Together, these requirements and models provide a clear foundation for subsequent design and implementation activities and ensure the system can support academic integrity review in a scalable and privacy-preserving manner.